Example Documentation

generated from github wiki: 17.12.2025

Contents

1 Einführung & Systemübersicht 3
1.1 Einführung 3
1.2 Ziel 3
1.3 Systemübersicht 3

2 Architektur 4
2.1 Logisch 4
2.1.1 Komponenten-Übersicht 4
2.1.2 Architektur-Muster 4
2.1.3 Modul-Interaktionen 4
2.1.4 Separation of Concerns 5

2.2 Physisch 5
2.2.1Hardware-Topologie 5

3 Programmablauf 6
3.1 Systemstart 7
3.2 Wartezustand (Idle) 7
3.3 Begrüssung 7
3.4 Themenauswahl 7
3.5 Geschichtenerzählung 7
3.6 Verabschiedung 7

4 Externe Lösungen und Algorithmen 8
4.1 Story & Logik: Google Gemini API (Cloud Service) 8
4.2 Sprachsynthese: Coqui TTS & FFmpeg 8
4.3 Wahrnehmung: Nuitrack Middleware (Lokale Middleware) 8
4.4 Algorithmen 8
A. Die State Machine (Logischer Ablauf) 8
B. Face-Tracking 8

1 Einführung & Systemübersicht

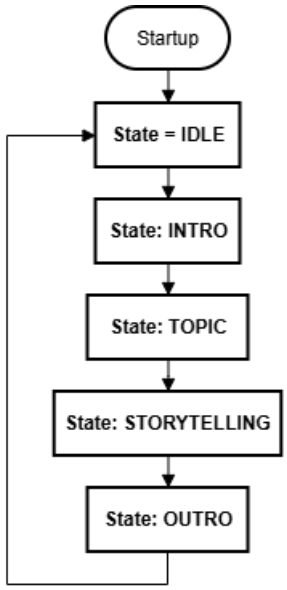
1.1 Einführung

Im HRI-Projekt "Interactive Storyteller" haben wir mithilfe des QTrobot ein System entwickelt, welches nicht nur eine Geschichte erzählt, sondern den Nutzer auch aktiv in den Erzählprozess einbindet. Das System erlaubt es Themenwünsche einzubringen und Entscheidungen über den weiteren Verlauf der Handlung zu treffen, sodass die Geschichte dynamisch an die Eingaben des Nutzers angepasst wird.

1.2 Ziel

Das Ziel des Projekts besteht darin, ein interaktives Erzählsystem zu entwickeln, das die technischen Möglichkeiten des QTrobot nutzt, um eine dynamische und nutzerzentrierte Geschichte zu erzeugen. Dabei soll untersucht werden, wie sich bestehende QT-Funktionen, wie zum Beispiel Gesichtstracking oder Sprachinteraktion, sinnvoll mit einer externen LLM-API kombinieren lassen, um eine flexible und natürlich wirkende Interaktion zu ermöglichen. Neben der technischen Umsetzung mussten dabei auch grundlegende Aspekte der Mensch-Roboter-Interaktion berücksichtigt werden. Dazu gehören unter anderem Überlegungen zum Roboterdesign (z. B. Individualisierung und verständliche Affordances), zur verbale und non-verbalen Kommunikation, zur räumlichen Interaktion sowie zur Sicherheit im Umgang mit dem Roboter.

1.3 Systemübersicht



1. **Startup:** Initialisiert alle benötigten Module (LLM, ROS, TTS, Face-Tracking, Speech-Recognition) und startet die `State Machine`.
2. **IDLE:** Roboter wartet, bis ein Gesicht erkannt wird.
3. **INTRO:** Kurze Begrüßung, bevor die Interaktion beginnt.
4. **TOPIC_SELECTION:** Nutzer wählt ein Thema für die Geschichte.
5. **STORYTELLING:** Die Geschichte wird in mehreren Phasen erzählt. Das LLM liefert Story-Abschnitte und Auswahlmöglichkeiten, der Nutzer beeinflusst den Verlauf.
6. **OUTRO:** Der Roboter beendet die Geschichte und kehrt in den `IDLE`-Zustand zurück.

2 Architektur

Dieses Kapitel beschreibt zunächst die logische Struktur mit ihren Komponenten, Mustern und Interaktionen, anschliessend die physische Verteilung auf Hardware-Komponenten und Netzwerk-Topologie.

2.1 Logisch

Unsere Storytelling-Anwendung folgt einer zustandsbasierten Architektur mit klarer Trennung von Geschäftslogik und technischen Services. Das System orchestriert KI-generierte Geschichten, Roboter-Interaktionen und Benutzereingaben über eine zentrale State Machine.

2.1.1 Komponenten-Übersicht

Die Architektur gliedert sich in drei Hauptschichten:

1. Core-Schicht (Geschäftslogik):

- **StorytellerStateMachine:** Zentrale Orchestrierung der Anwendung. Verwaltet fünf Zustände (IDLE, INTRO, TOPIC_SELECTION, STORYTELLING, OUTRO) und delegiert die Ausführung an spezialisierte Handler.
- **State Handlers:** Fünf Handler implementieren zustandsspezifisches Verhalten. Der `StorytellingHandler` ist das Herzstück und koordiniert Story-Generierung, parallele Audio-Verarbeitung und Face-Tracking.
- **StoryContext:** Shared Memory für Topic, Story-Historie und Benutzer-Entscheidungen. Wird zwischen allen Handlers geteilt.

2. Service-Schicht (Technische Abstraktion):

- **LLMService:** Kommunikation mit Google Gemini API (`gemini-2.5-flash-lite`). Baut strukturierte Prompts auf und parst JSON-Antworten mit Story-Chunks, Emotionen und Auswahloptionen.
- **RobotService:** Fassade für alle Roboter-Funktionen. Implementiert lokale TTS-Generierung (Coqui TTS, Modell vctk/vits), Audio-Nachbearbeitung (ffmpeg) und koordiniert ROS-Publisher für Emotionen/Gesten mit ROS-Services für Audio-Wiedergabe.
- **InputService:** Abstrahiert Spracheingabe über Custom ROS Service für Speech-to-Text.
- **FaceTrackerService:** P-Controller für Kopfbewegungen. Nutzt Nuitrack-Gesichtsdaten und publiziert Pan/Tilt-Befehle. Sammelt parallel Emotionsdaten während des Trackings.

3. Infrastruktur-Schicht:

- **PlayAudioFileServer:** Separater ROS-Node für SSH-basierten Audio-Transfer zur Roboter-Peripherie. Überbrückt Container-Umgebung und physische Hardware. Die Implementierung dieser ROS-Node wurde vom Dozenten zur Verfügung gestellt.

2.1.2 Architektur-Muster

State Pattern: Die `StorytellerStateMachine` implementiert das klassische State Pattern. Jeder Handler definiert seine eigene `handle()`-Methode und gibt den Folgezustand zurück. Dies ermöglicht klare Zustandsübergänge ohne verschachtelte Conditionals.

Dependency Injection: Alle Services werden zentral in der State Machine instanziiert und an Handler weitergereicht. Dies erleichtert Testing und vermeidet redundante Initialisierungen (z.B. TTS-Modell-Loading).

Producer-Consumer: Der `RobotService` nutzt eine Queue für parallele Audio-Generierung. Während Story-Chunks generiert werden, läuft bereits die TTS-Synthese im Hintergrund. Dies reduziert wahrnehmbare Latenz erheblich.

Facade Pattern: Services kapseln komplexe Subsysteme. Der `RobotService` verbirgt z.B. die Komplexität von ROS-Kommunikation, TTS-Pipeline und Audio-Nachbearbeitung hinter einfachen Methoden wie `parallel_output()`.

2.1.3 Modul-Interaktionen

Der typische Story-Ablauf zeigt das Zusammenspiel:

1. **Story-Generierung:** Handler erstellt Prompt → LLMService ruft Gemini API auf → JSON-Response mit story_chunks wird geparst → Chunks landen im StoryContext
2. **Parallele Verarbeitung:** Für jeden Chunk wird Audio-Generierung gestartet → WAV-Dateien werden in Queue gelegt → Handler startet Face-Tracking
3. **Ausgabe:** Audio wird aus Queue gelesen → PlayAudioFileServer transferiert via SSH → Parallel werden Emotionen/Gesten über ROS-Publisher gesendet
4. **Benutzereingabe:** InputService ruft Speech-Recognition-Service auf → Transkription wird im Context gespeichert → Nächster Chunk wird basierend auf Auswahl generiert

2.1.4 Separation of Concerns

Die Architektur trennt konsequent Verantwortlichkeiten:

- **Vertikal (nach Schicht):** Core-Schicht kennt keine ROS-Details. Services kennen keine Story-Logik. Handler orchestrieren nur, implementieren aber keine technischen Details.
- **Horizontal (nach Funktion):** LLM-Prompts (`prompt.py`) sind getrennt von API-Kommunikation. Audio-Koordination ist getrennt von TTS-Generierung und Transfer.
- **Daten vs. Logik:** `StoryContext` ist reines Datenobjekt ohne Geschäftslogik. Services sind zustandslos (außer technischer Caches).

2.2 Physisch

Unsere Anwendung folgt einer zentralisierten ROS-Knoten-Architektur, die durch Cloud-Ressourcen erweitert wird. Wir unterscheiden dabei zwischen der lokalen Verarbeitung auf dem Roboter (für Latenz-kritische Aufgaben und Datenschutz) und der externen Verarbeitung (für rechenintensive KI).

2.2.1Hardware-Topologie

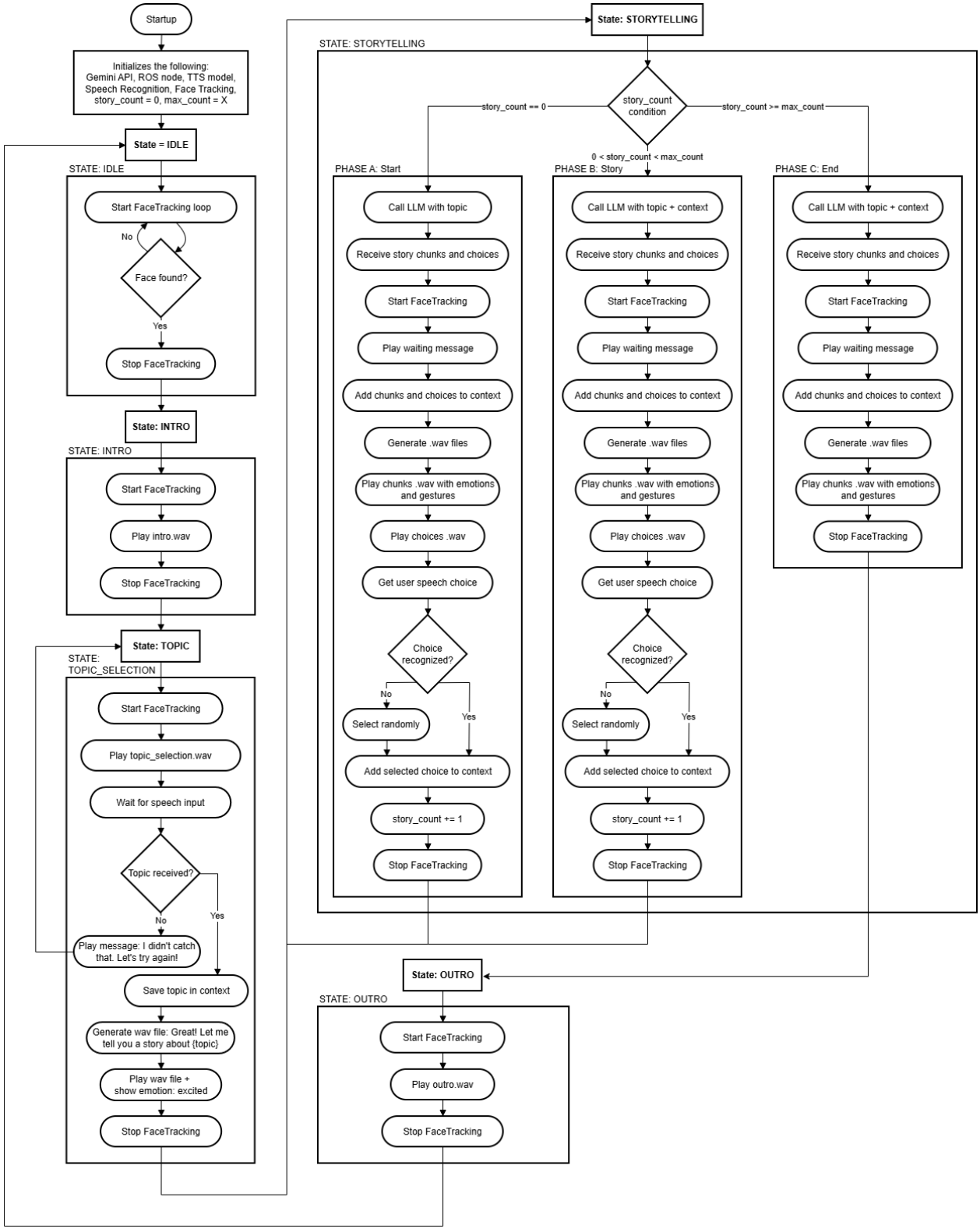
Die physische Verteilung der Aufgaben gliedert sich in zwei Hauptkomponenten:

1. **QTRobot PC:** Dies ist das Herzstück unserer physischen Architektur, in dem Fall der Container, den wir benutzen konnten. Hier läuft der ROS-Master und unser primärer Applikations-Knoten. Dieser hostet die `StorytellerStateMachine`, die Zustandsverwaltung, die lokale TTS-Generierung (Coqui) und das Head-Tracking. Er nutzt CPU und RAM intensiv für die Audio-Synthese und Bildverarbeitung.

2. **QTRobot Peripherie & Controller:** Die Sensoren und Aktoren des QTRobots.

- Kamera & Nuitrack: Die Middleware Nuitrack läuft als Hintergrunddienst und liefert vorverarbeitete Skelett-Daten an unseren Knoten.
- Mikrofon & Audio: Unser `InputService` greift über ROS-Services auf die Audio-Hardware zu.
- Motoren: Der `RobotService` und `FaceTrackerService` sendet Befehle an die Motor-Controller (Head/Arms).

3 Programmablauf



Das Flowchart bildet den gesamten Ablauf des interaktiven Storytelling-Systems ab. Es zeigt die fünf zentralen Zustände, sowie die Übergänge, der `State Machine`: Startup, `IDLE`, `INTRO`, `TOPIC_SELECTION`, `STORYTELLING` und `OUTRO`. Der Prozess beginnt mit der Initialisierung sämtlicher Systemkomponenten und führt anschließend über eine ereignisgesteuerte Logik durch die Interaktion mit dem Nutzer. Das Diagramm verdeutlicht, wie Gesichtserkennung, Audioausgabe, Sprachverarbeitung, LLM-Anfragen und Kontextverwaltung ineinandergreifen, um eine dynamische und adaptive Geschichte zu erzeugen. Nach Abschluss einer Story kehrt das System wieder in den `IDLE`-Zustand zurück und ist bereit für eine neue Interaktion.

3.1 Systemstart

Im Startup-Prozess wird zunächst die `StorytellerStateMachine` erzeugt, welche die Verwaltung aller Zustände übernimmt. Anschliessend initialisiert das System die Gemini API, startet den ROS-Node, lädt das TTS-Modell, richtet den Speech-Recognition-Proxy ein und konfiguriert die Nuitrack-Face-Tracking-Parameter. Zusätzlich wird der Story-Zähler gesetzt und der initiale Zustand auf `IDLE` gestellt. Danach beginnt eine Endlosschleife, in der die `State Machine` kontinuierlich ausgeführt wird.

3.2 Wartezustand (Idle)

Im `IDLE`-Zustand wartet das System passiv auf eine Person. Ein kontinuierlicher Face-Tracking-Loop läuft so lange, bis ein Gesicht erkannt wird. Sobald dies geschieht, stoppt das System das Tracking und wechselt in den `INTRO`-Zustand. Wird kein Gesicht erkannt, verbleibt das System dauerhaft in dieser Schleife.

3.3 Begrüssung

Der `INTRO`-Zustand dient als Begrüssungsphase. Das System startet erneut das Face-Tracking, spielt eine vor aufgezeichnete Intro-Audiodatei ab und beendet anschliessend das Tracking. Danach erfolgt der Übergang in den Zustand `TOPIC_SELECTION`, in dem die eigentliche Interaktion beginnt.

3.4 Themenauswahl

In der `TOPIC_SELECTION` fordert der Roboter den Nutzer auf, ein Thema für die Geschichte zu nennen. Dazu startet das System das Face-Tracking und spielt eine vor aufgezeichnete Audioanweisung ab. Anschliessend wird die Spracherkennung aktiviert und das System wartet auf eine Antwort. Wird ein Thema erkannt, wird es im Kontext gespeichert. Falls die Spracherkennung scheitert, gibt der Roboter eine Fehlermeldung aus und wiederholt den Zustand. Sobald ein gültiges Thema vorliegt, erzeugt das System eine kurze TTS-Ausgabe, in der das gewählte Thema bestätigt wird und begleitet diese mit einer passenden Emotion. Danach wird das Face-Tracking beendet und das System wechselt in den `STORYTELLING`-Zustand.

3.5 Geschichtenerzählung

Der `STORYTELLING`-Zustand bildet den Kern des Systems und ist in drei Phasen unterteilt, die abhängig vom aktuellen Story-Zähler durchlaufen werden. In der ersten Phase, die nur beim Start der Geschichte ausgeführt wird, erzeugt das LLM auf Basis des gewählten Themas eine erste Struktur aus Story-Abschnitten und Auswahlmöglichkeiten. Während das System eine kurze Wartebotschaft abspielt, werden die Inhalte in den Kontext übernommen und alle benötigten Audiodateien generiert. Die Story-Abschnitte werden anschliessend mit passenden Emotionen und Gesten präsentiert. Danach hört das System auf die Nutzerwahl; falls keine eindeutige Antwort erkannt wird, wird eine zufällige Option gewählt. Der Story-Zähler wird erhöht und das System kehrt in den `STORYTELLING`-Zustand zurück.

In der fortlaufenden Phase wird derselbe Ablauf wiederholt, jedoch unter Einbezug der bisherigen Story-Historie. Das LLM erhält also den gesamten bisherigen Kontext, um eine konsistente Fortsetzung zu erzeugen. Auch hier werden die neuen Story-Abschnitte präsentiert, die Nutzerwahl abgefragt und der Story-Zähler erhöht, bevor das System erneut in den `STORYTELLING`-Zustand zurückkehrt.

Sobald der Story-Zähler den definierten Maximalwert erreicht, beginnt die finale Phase. Das LLM erzeugt einen abschliessenden Story-Abschnitt ohne weitere Auswahlmöglichkeiten. Die finalen Inhalte werden wie zuvor verarbeitet, mit Emotionen und Gesten präsentiert und anschliessend wird das Face-Tracking beendet. Danach wechselt das System in den `OUTRO`-Zustand.

3.6 Verabschiedung

Im `OUTRO`-Zustand verabschiedet sich das System von dem Nutzer. Dazu startet es das Face-Tracking, spielt eine vor aufgezeichnete Outro-Audiodatei ab und beendet das Tracking wieder. Anschliessend kehrt das System wieder in den `IDLE`-Zustand zurück. Damit beginnt der gesamte Ablauf erneut und das System ist bereit für eine weitere Geschichte.

4 Externe Lösungen und Algorithmen

Um die Funktionalität unseres interaktiven Storytellings zu gewährleisten, greifen wir auf eine Kombination aus Cloud-Services und lokalen Bibliotheken zurück.

4.1 Story & Logik: Google Gemini API (Cloud Service)

Für die dynamische Generierung der Geschichte nutzen wir das Large Language Model (LLM) von Google.

- **Service:** Google Gemini API
- **Modell:** `gemini-2.5-flash-lite`
- **Verwendung:** Dieser Service fungiert als das "kreative Gehirn" des Roboters. Wir senden hier immer die gesamte Story und die getroffenen Entscheidungen des Users mit.
- **Technische Integration:**
  - Die Kommunikation erfolgt über eine **REST-Schnittstelle** (`requests`-Library).
  - Der Output wird als **JSON-Objekt** angefordert und geparst, um den reinen Text von den Emotionen und Gesten zu trennen.

4.2 Sprachsynthese: Coqui TTS & FFmpeg

Um dem Roboter eine angenehme, nicht-robotische Erzählstimme zu verleihen, benutzen wir ein TTS-Modell, welches lokal ausgeführt wird.

- **Bibliothek:** Coqui TTS
- **Modell:** `tts_models/en/vctk/vits`
- **Verwendung:** Umwandlung der generierten Text-Antworten in Audio-Dateien (`.wav`).
- **Technische Integration:**
  - Als Sprecher-ID wurde **`p230`** (männliche Stimme) gewählt, da diese eine angenehme Tiefe für das Storytelling besitzt.
  - **Audio-Post-Processing (FFmpeg):** Die generierten Audio-Dateien werden direkt im Anschluss durch `FFmpeg` (via `subprocess`) bearbeitet. Dabei reduzieren wir die Geschwindigkeit (`atempo=0.85`) und erhöhen die Lautstärke (`volume=5dB`), um eine ruhigere, verständlichere Erzählweise zu erreichen.

4.3 Wahrnehmung: Nuitrack Middleware (Lokale Middleware)

Für die visuelle Interaktion greifen wir auf die im QTRobot integrierte Middleware zurück.

- **Software:** Nuitrack SDK
- **Verwendung:** Erkennung des Benutzers und dessen Emotionen.
- **Funktion:** Nuitrack liefert uns die **Skelett-Daten** und **Gesichtsparameter**. Diese Rohdaten nutzen wir im `FaceTrackerService`, um festzustellen, ob eine Person anwesend ist und diese dann zu tracken.

4.4 Algorithmen

Unser Hauptprogramm ist ein einzelner ROS-Knoten, der in einer Endlosschleife (`while True`) eine **Finite State Machine (FSM)** ausführt.

A. Die State Machine (Logischer Ablauf)

Die Klasse `StorytellerStateMachine` koordiniert den physischen Ablauf. Sie hält den `StoryContext` (die bisherige Geschichte) und delegiert die Ausführung an spezialisierte Handler (`Intro`, `Storytelling`, `Outro`).

B. Face-Tracking

Damit der Roboter den Zuhörer während der Interaktion natürlich ansieht, implementieren wir einen eigenen Regelkreis im `FaceTrackerService`.

- **Funktionsweise:**
  1. Empfang der `Faces`-Nachricht von Nuitrack.
  2. Berechnung des Mittelpunkts zwischen den Augen (`eye_mid`).
  3. Berechnung des Fehlers zur Bildmitte (`error = eye_mid - center`).
  4. Berechnung der Stellgrösse für die Kopfmotoren: `command = Gain * error`.